CIS 4020 · DATA STRUCTURES

Red-Black Tree Deletion, Step by Step

Every node named, and every step spelled out

This handout goes with the Deletion section of *My Notes: Red-Black Trees*. It describes the same bottom-up deletion as the notes and the Java and C++ code, but it gives every node involved a name and uses only those names. A name stays attached to the same node until a step tells you to re-label.

1. The names

Table 1: The names used for the nodes in a deletion.

Name	Which node it is
D	The node holding the value you were asked to delete.
S	D's inorder successor: the leftmost node in D's right subtree. S is used only when D has two children. S never has a left child.
R	The node that is actually taken out of the tree. R is D when D has zero children or one child, and R is S when D has two children.
C	Whatever moves into R's old position when R is taken out: R's only child, or an **empty spot** if R had no child. An empty spot counts as black.
P	C's parent.
W	C's sibling, which is P's other child.
N	W's **near** child: the child of W on the same side as C. If C is a left child, N is W's left child.
F	W's **far** child: W's other child.

A missing child of W also counts as black, so N or F can be an empty spot too.

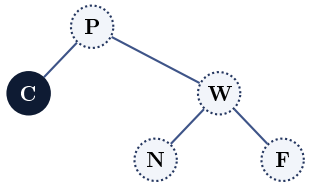


Figure 1: C is a left child. N, the near child, is W's left child, and F, the far child, is W's right child. Root P (either color). Level 2: C (black), W (either color). Level 3: N (either color), F (either color).

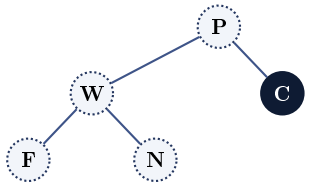


Figure 2: C is a right child, the mirror image. N is W's right child, and F is W's left child. Root P (either color). Level 2: W (either color), C (black). Level 3: F (either color), N (either color).

Two phrases in the steps tell you which way to rotate.

- **Rotate at P toward C**: rotate left at P if C is P's left child, and rotate right at P if C is P's right child. P moves down on C's side.
- **Rotate at W away from C**: rotate right at W if C is P's left child, and rotate left at W if C

is P's right child.



Figure 3: How the figures are drawn. A black node has a dark fill with white text. A red node has a white fill and a thick dashed outline. A node of either color has a light fill and a dotted outline. An empty spot is a small dark square on a dashed line. The names from Table 1 are printed in bold next to the nodes they belong to.

2. The procedure

Stage A: remove a node

Step 1. Find D, the node holding the value to delete.

Step 2. Decide which node is R.

- If D has **zero children or one child**, R is D.
- If D has **two children**, find S: go to D's right child, then go left as far as possible. Copy S's value into D. **D keeps D's own color**; only the value stored in D changes. R is S, the node lower down that still holds the copied value.

Step 3. Write down **R's color**. This is the color that decides whether a fix-up is needed. D's color does not matter here, unless R is D.

Step 4. Take R out of the tree. If R has a child, that child moves up into R's position under R's parent. If R has no child, R's position becomes an empty spot. Whatever is now in R's old position is **C**, and R's old parent is **P**.

Stage B: decide whether a fix-up is needed

Step 5. If R was **red**, stop. Taking out a red node does not change the number of black nodes on any path.

Step 6. If R was black and C is **red**, color C black and stop. C now replaces the black node that was lost. This happens only when R had exactly one child.

Step 7. Otherwise, R was black and C is black or an empty spot. Every path through C is now one black node short. Say that C carries an **extra black**, and go to Stage C.

Stage C: the fix-up

Start at Step 8. Check the cases in order; exactly one of them applies each time. Steps 9, 10, and 11 each say where to go next.

Step 8. If C is the root of the tree, color C black and stop. Otherwise, find P, W, N, and F for the current C.

Step 9. **Case 1, W is red.** (P must be black, since P has a red child.)

- Color W black and color P red.
- Rotate at P toward C. W moves up into P's old position. P moves down and is still C's parent. N, which is black, becomes P's child on the side away from C.

- Re-label: N is now C's sibling, so N is the new **W**. Find the new N and F as the children of the new W.
- The new W is black, so go to Step 10.

Step 10. Case 2, W is black, and N and F are both black.

- Color W red. The extra black moves from C up to P.
- If P is red, color P black and stop.
- If P is black, P now carries the extra black. Re-label: P is the new **C**. Go back to Step 8, one level higher in the tree.

If you are here right after Case 1, P is red, so you always stop.

Step 11. Case 3, W is black, N is red, and F is black.

- Color N black and color W red.
- Rotate at W away from C. N moves up into W's old position, and W becomes N's child on the side away from C.
- Re-label: the old N is the new **W**, and the old W, which is red, is the new **F**. The new N is the new W's child on C's side.
- The new F is red, so go to Step 12.

Step 12. Case 4, W is black and F is red. N can be either color.

- Give W the color that P has right now.
- Color P black and color F black.
- Rotate at P toward C. W moves up into P's old position, P moves down to W's side toward C with C still below it, and N becomes P's child on the side away from C.
- Stop. The extra black is gone.

Only Case 2 can send you back up the tree. Cases 1, 3, and 4 each happen at most once in a deletion, so a deletion does at most three rotations.

3. The four cases, drawn with the names

Each case is drawn with C as P's left child. When C is P's right child, every picture is the mirror image, and every left becomes right.

Case 1: W is red

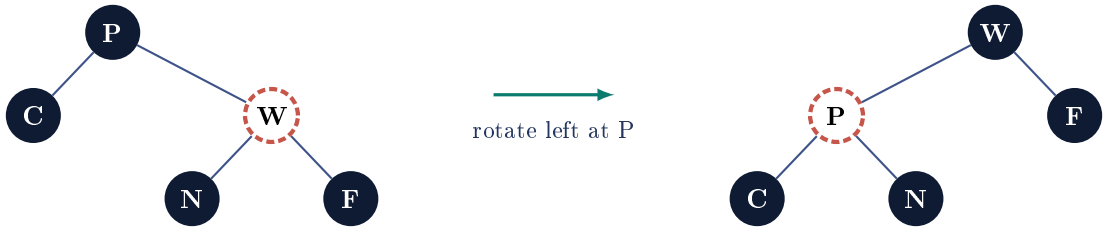


Figure 4: Case 1 with C as a left child. Before: black P with black C on the left and red W on the right; W has black children N and F. After: black W on top; its left child P is red with children C and N; F stays on the right. C's new sibling is the old N.

Case 2: W is black, and N and F are both black

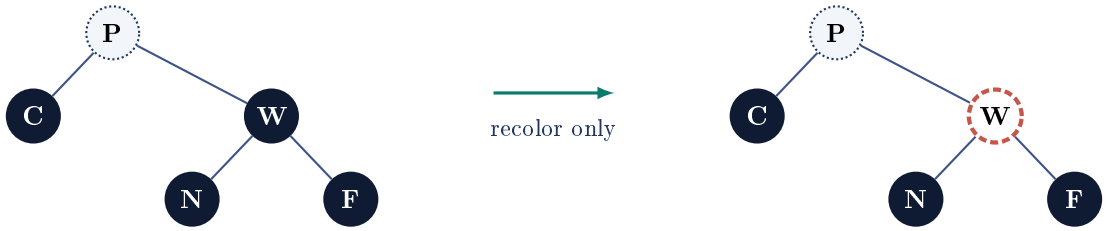


Figure 5: Case 2 with C as a left child. Before: P of either color, black C, and black W with black children N and F. After: the same tree with W red. The extra black has moved up to P.

Case 3: W is black, N is red, and F is black

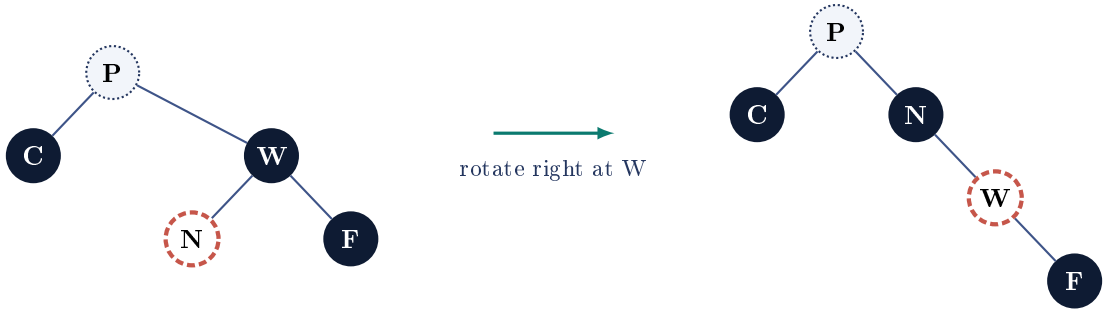


Figure 6: Case 3 with C as a left child. Before: P of either color, black C, and black W whose near child N is red and far child F is black. After: N is black and is now C's sibling; W is N's red right child, with F below it.

After the rotation, re-label before going on: the node drawn as N is the new W, and the node drawn as W is the new F.

Case 4: W is black and F is red

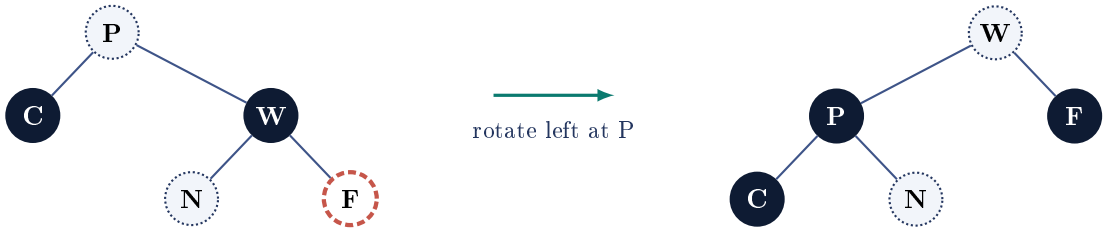


Figure 7: Case 4 with C as a left child. Before: P of either color, black C, and black W whose far child F is red; N may be either color. After: W is on top with P's old color; P is black with children C and N; F is black.

4. Worked examples

These are the five deletions from *My Notes: Red-Black Trees*, in the same order, so each one starts from the tree the one before it left. A sixth example, from the same starting tree, shows a node with one child.

Delete 85: Case 2, and the fix-up stops at P

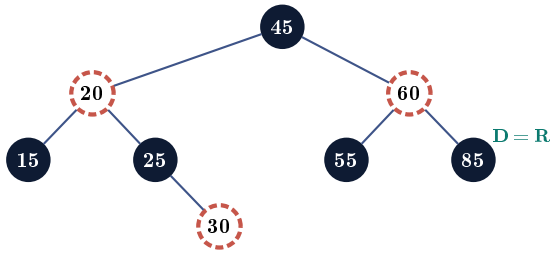


Figure 8: Delete 85, before. D and R are both 85. Root 45 (black). Level 2: 20 (red), 60 (red). Level 3: 15 (black), 25 (black), 55 (black), 85 (black, D = R). Level 4: 30 (red).

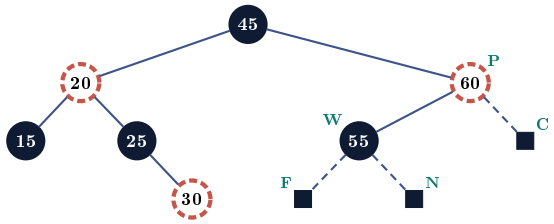


Figure 9: After step 4: R is gone and C is an empty spot. Root 45 (black). Level 2: 20 (red), 60 (red, P). Level 3: 15 (black), 25 (black), 55 (black, W), an empty spot (C). Level 4: 30 (red), an empty spot (F), an empty spot (N).

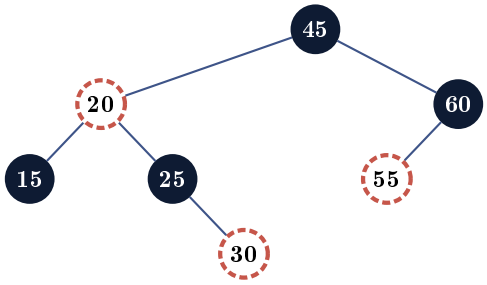


Figure 10: After deleting 85. Root 45 (black). Level 2: 20 (red), 60 (black). Level 3: 15 (black), 25 (black), 55 (red). Level 4: 30 (red).

Steps 1 to 3

D is the node holding 85. D has no children, so R is D. R is black.

Step 4

R had no child, so R's position, the right child of 60, becomes an empty spot. That empty spot is C, and P is 60.

Steps 5 to 7

R was black, and C is an empty spot, which counts as black. C carries the extra black.

Step 8

C is not the root. W is 55, P's other child. C is a right child, so N is W's right child and F is W's left child. Both are empty spots, so both count as black.

Step 9

W is black, so this is not Case 1.

Step 10

W is black, and N and F are black: Case 2. Color W (55) red. P (60) is red, so color P black and stop.

Delete 45: two children, and R is red

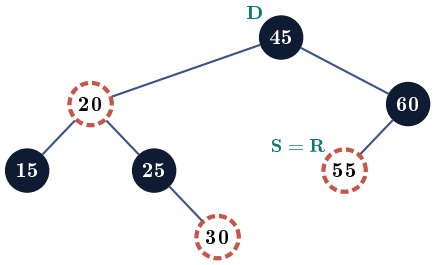


Figure 11: Delete 45, before. D is 45; S, which is also R, is 55. Root 45 (black, D). Level 2: 20 (red), 60 (black). Level 3: 15 (black), 25 (black), 55 (red, S = R). Level 4: 30 (red).

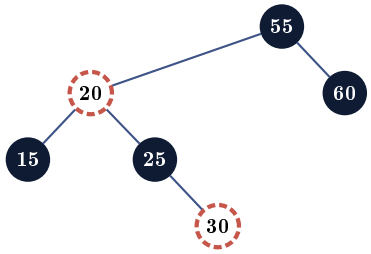


Figure 12: After deleting 45. The root still has D's color, black, and now holds 55. Root 55 (black). Level 2: 20 (red), 60 (black). Level 3: 15 (black), 25 (black). Level 4: 30 (red).

Step 1

D is the root, which holds 45.

Step 2

D has two children. To find S, go to D's right child, 60, then go left as far as possible: 60's left child is 55, and 55 has no left child, so S is 55. Copy 55 into D. D keeps D's color, black, so the root now holds 55 and is still black. R is S, the node lower down that still holds 55.

Step 3

R is red.

Step 4

R had no child, so 60's left child becomes an empty spot. That is C, and P is 60.

Step 5

R was red, so stop.

Delete 15: Case 4

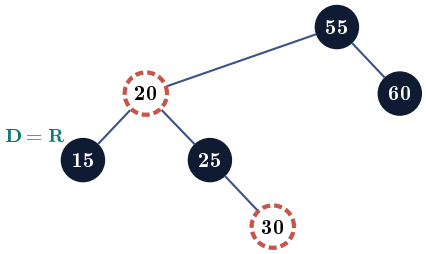


Figure 13: Delete 15, before. D and R are both 15. Root 55 (black). Level 2: 20 (red), 60 (black). Level 3: 15 (black, D = R), 25 (black). Level 4: 30 (red).

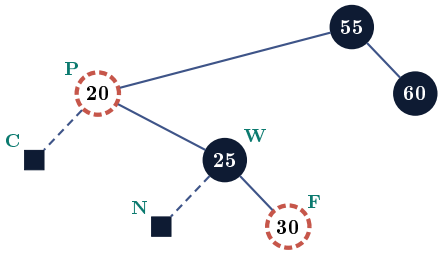


Figure 14: After step 4: C is an empty spot. Root 55 (black). Level 2: 20 (red, P), 60 (black). Level 3: an empty spot (C), 25 (black, W). Level 4: an empty spot (N), 30 (red, F).

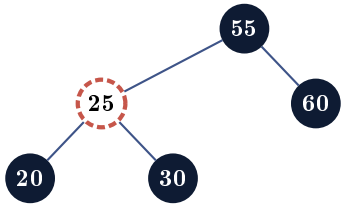


Figure 15: After Case 4, done. Root 55 (black). Level 2: 25 (red), 60 (black). Level 3: 20 (black), 30 (black).

Steps 1 to 3

D is the node holding 15. D has no children, so R is D. R is black.

Step 4

R's position, the left child of 20, becomes an empty spot. That is C, and P is 20.

Steps 5 to 7

R was black, and C is an empty spot. C carries the extra black.

Step 8

C is not the root. W is 25. C is a left child, so N is W's left child, an empty spot, and F is W's right child, 30.

Steps 9 to 11

W is black, so not Case 1. F is red, so not Case 2. N is black, so not Case 3.

Step 12

Case 4. Give W (25) the color of P (20), red. Color P (20) black and F (30) black. C is a left child, so rotate left at P (20). W (25) moves up into 20's old position. Stop.

Delete 60: Case 1, then Case 2

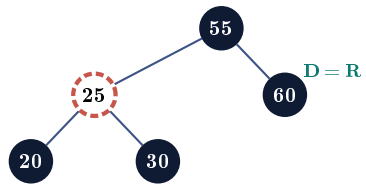


Figure 16: Delete 60, before. D and R are both 60. Root 55 (black). Level 2: 25 (red), 60 (black, D = R). Level 3: 20 (black), 30 (black).

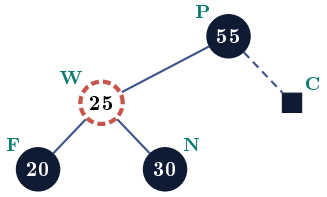


Figure 17: After step 4: C is an empty spot on the right. Root 55 (black, P). Level 2: 25 (red, W), an empty spot (C). Level 3: 20 (black, F), 30 (black, N).

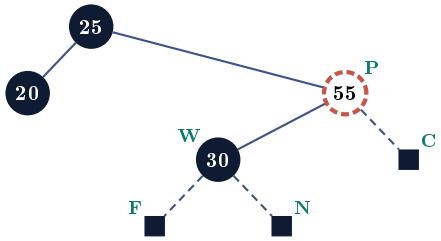


Figure 18: After Case 1, with the names re-labeled. Root 25 (black). Level 2: 20 (black), 55 (red, P). Level 3: 30 (black, W), an empty spot (C). Level 4: an empty spot (F), an empty spot (N).

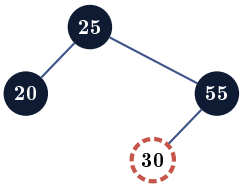


Figure 19: After Case 2, done. Root 25 (black). Level 2: 20 (black), 55 (black). Level 3: 30 (red).

Steps 1 to 3

D is the node holding 60. D has no children, so R is D. R is black.

Step 4

R's position, the right child of 55, becomes an empty spot. That is C, and P is 55.

Steps 5 to 7

R was black, and C is an empty spot. C carries the extra black.

Step 8

C is not the root. W is 25. C is a right child, so N is W's right child, 30, and F is W's left child, 20.

Step 9

W (25) is red: Case 1. Color W (25) black and P (55) red. C is a right child, so rotate right at P (55). 25 moves up to the root. P (55) is still C's parent, and the old N (30) becomes 55's left child. Re-label: the new W is 30. C is still a right child, so the new N is 30's right child and the new F is 30's left child. Both are empty spots.

Step 10

W (30) is black, and N and F are black: Case 2. Color W (30) red. P (55) is red, so color P black and stop.

Delete 20: Case 3, then Case 4

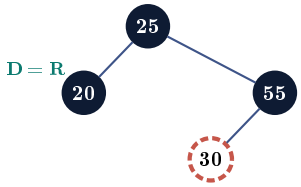


Figure 20: Delete 20, before. D and R are both 20. Root 25 (black). Level 2: 20 (black, D = R), 55 (black). Level 3: 30 (red).

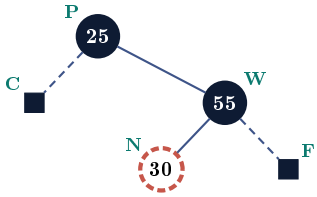


Figure 21: After step 4: C is an empty spot on the left. Root 25 (black, P). Level 2: an empty spot (C), 55 (black, W). Level 3: 30 (red, N), an empty spot (F).

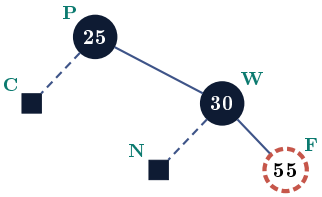


Figure 22: After Case 3, with the names re-labeled. Root 25 (black, P). Level 2: an empty spot (C), 30 (black, W). Level 3: an empty spot (N), 55 (red, F).

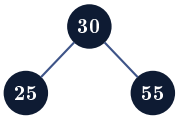


Figure 23: After Case 4, done. Root 30 (black). Level 2: 25 (black), 55 (black).

Steps 1 to 3

D is the node holding 20. D has no children, so R is D. R is black.

Step 4

R's position, the left child of 25, becomes an empty spot. That is C, and P is 25.

Steps 5 to 7

R was black, and C is an empty spot. C carries the extra black.

Step 8

C is not the root. W is 55. C is a left child, so N is W's left child, 30, and F is W's right child, an empty spot.

Steps 9 and 10

W is black, so not Case 1. N is red, so not Case 2.

Step 11

N is red and F is black: Case 3. Color N (30) black and W (55) red. C is a left child, so rotate right at W (55). 30 moves up into 55's old position, and 55 becomes 30's right child. Re-label: the new W is 30, the new F is 55, and the new N is 30's left child, an empty spot.

Step 12

F (55) is red: Case 4. Give W (30) the color of P (25), black. Color P (25) black and F (55) black. C is a left child, so rotate left at P (25). W (30) moves up to the root. Stop.

One more: a node with one child

This example starts again from the starting tree and deletes 25, which has one child.

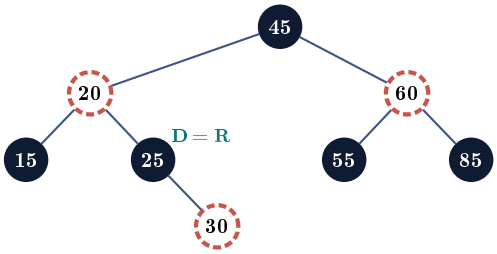


Figure 24: Delete 25 from the starting tree, before. D and R are both 25. Root 45 (black). Level 2: 20 (red), 60 (red). Level 3: 15 (black), 25 (black, D = R), 55 (black), 85 (black). Level 4: 30 (red).

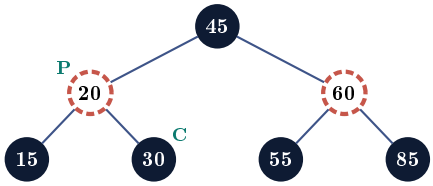


Figure 25: After deleting 25. C, which is 30, moved up and turned black. Root 45 (black). Level 2: 20 (red, P), 60 (red). Level 3: 15 (black), 30 (black, C), 55 (black), 85 (black).

Steps 1 to 3
D is the node holding 25. D has one child, 30, so R is D. R is black.

Step 4
R's only child, 30, moves up into R's position, the right child of 20. C is 30, and P is 20.

Step 5
R was black, so go on.

Step 6
C (30) is red, so color C black and stop.

5. Matching the names to the code

Table 2: The names in this handout and the variables in `RedBlackTree.java` and `RedBlackTree.cpp`.

Name	Variable	Notes
D	`z`	The node found by the search at the start of `delete` or `remove`.
S, R	`y`	`y` starts as `z` and becomes the successor when `z` has two children. `removedColor` holds R's color.
C	`x`	When R has no child, `x` is `NIL`, the shared black node that stands for every empty spot.
P	`x.parent`	`x->parent` in C++.
W	`sib`	
N, F	`sib.left`, `sib.right`	Which one is near depends on whether `x` is a left child or a right child.

One difference: instead of copying S's value into D, the code moves S's whole node up into D's position and gives it D's color (`y.color = z.color`). The finished tree has the same values in the same positions with the same colors, so the two descriptions give the same result.